

33. Multi-Channel Decimation for Massive MIMO

Simulate multi-channel decimation in Massive MIMO receiver and analyze computational reduction. (25) (BL4) (WK4)

Given Data:

Number of Channels = 16

Sampling Rate = 122.88 MHz

Decimation Factor = 4

Tasks:

Generate multi-channel signals.

Design a common decimation filter.

Perform decimation.

Analyze computational reduction.

Compare channel spectra.

Discuss implementation benefits.

MATLAB Code:

```
clc;
```

```
clear;
```

```
close all;
```

```
%% Multi-Channel Decimation for Massive MIMO
```

```
% Given Data
```

```
Nch = 16;           % Number of channels
```

```
Fs = 122.88e6;      % Sampling frequency = 122.88 MHz
```

```
M = 4;             % Decimation factor
```

```
Fout = Fs/M;        % Output sampling frequency
```

```
%% 1. Generate Multi-Channel Signals
```

```
T = 1e-4;           % Simulation time
```

```
t = 0:1/Fs:T-1/Fs;
```

```
N = length(t);
```

```
% Allocate input signal matrix
```

```
x = zeros(Nch,N);
```

```

% Different frequency for each channel
channelFreq = linspace(5e6,40e6,Nch);

for ch = 1:Nch
    % Multi-channel sinusoidal signal
    x(ch,:) = sin(2*pi*channelFreq(ch)*t);

    % Add small noise
    x(ch,:) = x(ch,:) + 0.05*randn(1,N);
end

fprintf('Number of Channels    = %d\n',Nch);
fprintf('Input Sampling Rate    = %.2f MHz\n',Fs/1e6);
fprintf('Decimation Factor        = %d\n',M);
fprintf('Output Sampling Rate     = %.2f MHz\n',Fout/1e6);

```

%% 2. Design a Common Decimation Filter

```

% FIR low-pass filter
filterOrder = 63;

% Cutoff frequency
Fc = Fout/2;

% Normalized cutoff frequency
Wn = Fc/(Fs/2);

% Common filter used for all channels
b = fir1(filterOrder,Wn);

fprintf('Common FIR Filter Order = %d\n',filterOrder);

```

%% 3. Perform Filtering and Decimation

```

y = zeros(Nch,ceil(N/M));

```

```
for ch = 1:Nch
```

```
    % Filter the signal
```

```
    filteredSignal = filter(b,1,x(ch,:));
```

```
    % Downsample by factor of 4
```

```
    y(ch,:) = filteredSignal(1:M:end);
```

```
end
```

%% 4. Computational Reduction Analysis

```
% Without decimation:
```

```
% Every input sample is processed at Fs
```

```
% With decimation:
```

```
% Only 1 out of every M samples is retained
```

```
originalSamples = N * Nch;
```

```
decimatedSamples = size(y,2) * Nch;
```

```
reductionPercentage = ...
```

```
    (1 - decimatedSamples/originalSamples)*100;
```

```
fprintf('\n----- Computational Analysis -----\n');
```

```
fprintf('Original samples processed = %d\n',originalSamples);
```

```
fprintf('After decimation      = %d\n',decimatedSamples);
```

```
fprintf('Computational reduction  = %.2f %%\n',reductionPercentage);
```

%% 5. Plot Input Signals of First 4 Channels

```
figure;
```

```
for ch = 1:4
```

```
    subplot(4,1,ch);
```

```
    plot(t(1:1000)*1e6,x(ch,1:1000));
```

```
    grid on;
```

```

    xlabel('Time (\mus)');
    ylabel('Amplitude');
    title(['Input Signal - Channel ',num2str(ch)]);
end

```

%% 6. Plot Output Signals of First 4 Channels

```

t_out = (0:size(y,2)-1)/Fout;

figure;

for ch = 1:4
    subplot(4,1,ch);
    plot(t_out(1:min(1000,length(t_out)))*1e6,...
        y(ch,1:min(1000,length(t_out))));
    grid on;
    xlabel('Time (\mus)');
    ylabel('Amplitude');
    title(['Decimated Signal - Channel ',num2str(ch)]);
end

```

%% 7. Compare Channel Spectra

```

figure;

for ch = 1:Nch

    % FFT of decimated signal
    Y = fft(y(ch,:));
    L = length(Y);

    P2 = abs(Y/L);
    P1 = P2(1:floor(L/2)+1);

    f = (0:floor(L/2))*Fout/L;

    subplot(4,4,ch);

```

```

plot(f/1e6,20*log10(P1+1e-12));
grid on;

xlabel('Frequency (MHz)');
ylabel('Magnitude (dB)');
title(['Channel ',num2str(ch)]);

end

sgtitle('Spectra of 16 Decimated MIMO Channels');

```

%% 8. Display Channel Frequencies

```

fprintf('\n----- Channel Frequency Information -----\n');

for ch = 1:Nch
    fprintf('Channel %2d : %.2f MHz\n', ...
        ch,channelFreq(ch)/1e6);
end

```

%% 9. Computational Benefit

```

fprintf('\n----- Implementation Benefits -----\n');
fprintf('1. Number of channels      : %d\n',Nch);
fprintf('2. Decimation factor        : %d\n',M);
fprintf('3. Sampling rate before       : %.2f MHz\n',Fs/1e6);
fprintf('4. Sampling rate after        : %.2f MHz\n',Fout/1e6);
fprintf('5. Data-rate reduction        : %.2f %%\n',reductionPercentage);
fprintf('6. Common filter is shared among all channels.\n');
fprintf('7. Lower processing and memory requirements.\n');
fprintf('8. Reduced communication bandwidth.\n');

```

OUTPUT:

